

Gradient descent, with a pencil

You have a function and you want its lowest point. You cannot see the whole surface — only the ground under your feet. Today you learn the entire method for finding the bottom anyway, and you run it five times by hand before you write a line of code.

Unit 1 · Foundations

80 minutes

Lab 6 · descend and plot

Last session before we build

[Slope](#)[The rule](#)[Five steps by hand](#)[In three lines](#)[The learning rate](#)[The chain rule](#)[Two omissions](#)[Lab 6](#)[Before Friday](#)

PART 1 · 10 MINUTES

Slope: which way is down, and how steeply

This is all the calculus this course needs, and you may already have it.

1

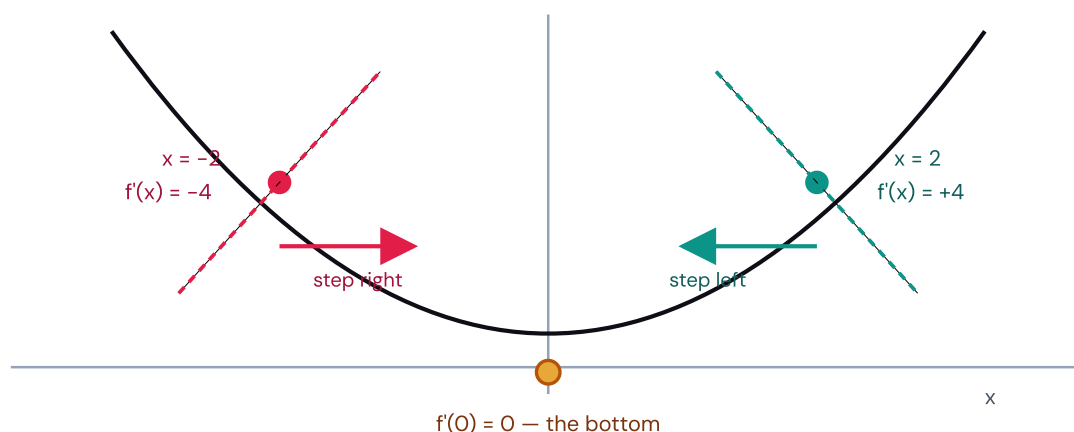
In Lab 1 you plotted $f(x) = x^2$. Look at it again. At any point on that curve there is a slope — how fast the curve rises as you move right. The slope is written $f'(x)$, and for this function it is:

THE ONE DERIVATIVE YOU MUST KNOW TODAY

$$f(x) = x^2 \implies f'(x) = 2x$$

At $x = 2$ the slope is 4: steep, going up to the right. At $x = -2$ the slope is -4: steep, going *down* to the right. At $x = 0$ the slope is 0, and that is the bottom.

THE SIGN OF THE SLOPE POINTS AWAY FROM THE MINIMUM — ALWAYS



Notice the pattern, because the whole method rests on it. Where the slope is **positive**, the minimum is to the **left**. Where the slope is **negative**, the minimum is to the **right**. In both cases, you should move in the direction *opposite* to the sign of the slope.

PART 2 · 6 MINUTES

The rule, in one line

Every model in this course, and every frontier model in the world, is trained by repeating this.

2

GRADIENT DESCENT

$$x \leftarrow x - \eta \cdot f'(x)$$

Take the slope. Move against it. The Greek letter η (eta) is the **learning rate** — how big a step to take. It is a number you choose, and Part 5 is about how much that choice matters.

The minus sign is the whole idea: it makes the step oppose the slope, so you always move downhill. And the size of the step is proportional to the slope, which gives you something valuable for free — **the steps get smaller as you approach the bottom**, because the slope gets smaller. You do not have to slow down manually. The method slows itself down.



Why not just solve $f'(x) = 0$?

For x^2 you could, and in session 10 you will do exactly that for a straight-line fit. But for a network with nine weights — let alone nine billion — there is no formula to solve. Gradient descent needs only the slope where you currently stand. It is the method that survives when the analytic answer does not exist, which is why it is the one that scaled.

PART 3 · 15 MINUTES · PENCILS OUT, LAPTOPS CLOSED

Five steps, by hand

Start at $x = 2$. Learning rate $\eta = 0.1$. Nobody types anything for the next fifteen minutes.

3

Substitute $f'(x) = 2x$ into the rule and something convenient happens:

the update simplifies

ARITHMETIC

$$\begin{aligned} x &\leftarrow x - \eta \cdot f'(x) \\ &= x - 0.1 \times 2x \\ &= x - 0.2x \\ &= 0.8x \end{aligned} \quad \leftarrow \text{every step just multiplies } x \text{ by } 0.8$$

So you can do this in your head. Fill the table in before looking at mine.

Step	x	$f(x) = x^2$	$f'(x) = 2x$	next $x = x - 0.1 \cdot f'(x)$
0	2.0000	4.0000	4.000	$2.0 - 0.4 = 1.6$
1	1.6000	2.5600	3.200	$1.6 - 0.32 = 1.28$
2	1.2800	1.6384	2.560	$1.28 - 0.256 = 1.024$
3	1.0240	1.0486	2.048	$1.024 - 0.2048 = 0.8192$
4	0.8192	0.6711	1.638	$0.8192 - 0.1638 = 0.6554$
5	0.6554	0.4295	1.311	...and so on



Three things to notice in your own table

$f(x)$ falls every single step

— 4.0, 2.56, 1.64, 1.05, 0.67, 0.43. That column is what a loss curve is.

The steps shrink

— 0.4, then 0.32, then 0.256. Nobody told it to slow down; the slope did that.

It never quite arrives.

After five steps $x = 0.655$, not 0. It approaches 0 and never reaches it. Training does not finish; you stop it. Session 16 is about deciding when.

PART 4 · 8 MINUTES

The same thing, in three lines

Now open the laptop, and check the machine against your paper.

4

the entire algorithm

PYTHON

```
def grad(x):  
    return 2 * x          #  $f'(x)$  for  $f(x) = x^2$   
  
x, eta = 2.0, 0.1  
for step in range(6):  
    print(f"step {step}: x = {x:.4f}    f(x) = {x*x:.4f}")  
    x = x - eta * grad(x)    ← the one line that matters
```

output — compare against your paper, digit by digit

OUTPUT

```
step 0: x = 2.0000    f(x) = 4.0000  
step 1: x = 1.6000    f(x) = 2.5600  
step 2: x = 1.2800    f(x) = 1.6384  
step 3: x = 1.0240    f(x) = 1.0486  
step 4: x = 0.8192    f(x) = 0.6711  
step 5: x = 0.6554    f(x) = 0.4295
```

YOU HAVE NOW WRITTEN A TRAINING LOOP

That is not a simplified version of one. In session 16 you will train a real network, and the loop will have the same three parts: compute the gradient, multiply by the learning rate, subtract. What changes is that x becomes thirteen numbers, then sixty thousand, then a hundred billion — and that computing the gradient stops being one line and becomes session 14.

The method does not change. It is this.

PART 5 · 16 MINUTES · THE HEART OF THE SESSION

The learning rate decides everything

And for this function, exactly one number tells you what will happen. You can compute it in your head.

5

Keep the rule general and see what one step does:

substitute $f'(x) = 2x$, but leave η as a symbol

ARITHMETIC

$$x \leftarrow x - \eta \cdot 2x = (1 - 2\eta) \cdot x$$

EVERY STEP MULTIPLIES x BY THE SAME FACTOR

$$\text{factor} = 1 - 2\eta$$

So the behaviour depends only on $|1 - 2\eta|$. Below 1, x shrinks toward zero. Above 1, it grows. Equal to 1, it does neither, forever. Negative, it flips sides each step.

You will not get a formula this clean again in this course. Use it now to build intuition you will keep.

η	$1 - 2\eta$	factor	What happens
0.01	+0.98	0.98	Converges — but agonisingly. See the trajectory below.
0.1	+0.80	0.80	Smooth and steady. The one you did by hand.
0.5	0.00	0.00	Lands exactly on the minimum in one step.
0.9	-0.80	0.80	Converges at the same rate as 0.1 — but overshoots and oscillates.
1.0	-1.00	1.00	Bounces between +2 and -2 forever. Never improves, never explodes.
1.1	-1.20	1.20	Diverges. Each step is further out than the last.

THE TRAJECTORIES, EIGHT STEPS FROM $x = 2$

run these yourself — the last row is the one to stare at

OUTPUT

$\eta=0.01$	2.000	1.960	1.921	1.882	1.845	1.808	1.772	1.736	1.702
$\eta=0.1$	2.000	1.600	1.280	1.024	0.819	0.655	0.524	0.419	0.336
$\eta=0.5$	2.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
$\eta=0.9$	2.000	-1.600	1.280	-1.024	0.819	-0.655	0.524	-0.419	0.336
$\eta=1.0$	2.000	-2.000	2.000	-2.000	2.000	-2.000	2.000	-2.000	2.000
$\eta=1.1$	2.000	-2.400	2.880	-3.456	4.147	-4.977	5.972	-7.166	8.600

Three questions about that table

Answer all three before opening. The second one catches almost everyone.

Q1. Why is $\eta = 0.5$ exact? Because $1 - 2(0.5) = 0$, so x is multiplied by zero. The step happens to be precisely the distance to the minimum. This is a coincidence of *this* function — do not expect it again, and never go looking for the "perfect" learning rate on a real problem.

Q2. $\eta = 0.1$ and $\eta = 0.9$ reach 0.336 after the same number of steps. Are they equally good? The magnitudes match exactly — both factors have size 0.80 — so on this function they converge at an identical rate. But 0.9 is *far* more dangerous. It sits close to the cliff at $\eta = 1.0$; a slightly steeper function, or one more layer, and it diverges. 0.1 has room. In practice you choose the stable side of equally-fast options, because you are never as certain about the curvature as you are here.

Q3. What is actually wrong with $\eta = 0.01$? Nothing, except time. It converges — 0.98 is below 1 — but after eight steps it has moved from 2.000 to 1.702. It would need roughly 340 steps to reach where $\eta = 0.1$ gets in 8. Multiply that by a network that takes a minute per pass and you have wasted a week to avoid a risk that was not there.



What divergence looks like when it happens to you

Look at the $\eta = 1.1$ row: 2.4, 2.88, 3.46, 4.15, 4.98... growing by 20% each step. In session 16 this will appear on your screen as a loss that climbs instead of falling, and then prints `nan`. When you see `nan`, your first move is to divide the learning rate by ten. Not to redesign the network. Not to get more data. Divide by ten and run it again.

PART 6 · 8 MINUTES

The chain rule — the second and last piece of calculus

You need it because a network is functions inside functions, and session 14 is nothing but this rule applied carefully.

6

Suppose a quantity depends on u , and u depends on x . How fast does the quantity change as x changes? You multiply the two rates.

THE CHAIN RULE

$$dy/dx = (dy/du) \times (du/dx)$$

If u changes 3 times as fast as x , and y changes 5 times as fast as u , then y changes 15 times as fast as x . That is the entire content of the rule.

worked, on a function you will meet again in session 11

ARITHMETIC

$$f(x) = (x - 3)^2$$

name the inside

$$\begin{aligned} u &= x - 3 & du/dx &= 1 \\ f &= u^2 & df/du &= 2u \end{aligned}$$

multiply the rates

$$df/dx = 2u \times 1 = 2(x - 3)$$

check at x = 0: 2(0-3) = -6 → slope is negative, minimum is to the right ✓

check at x = 5: 2(5-3) = +4 → slope is positive, minimum is to the left ✓



You can always check a derivative without trusting your algebra

Nudge x by a tiny amount and measure how much f moved:

```
h = 1e-6
numerical = (f(x + h) - f(x - h)) / (2 * h)
```

For $f(x) = (x-3)^2$ at $x = 0$ this gives -6.0000 , matching the algebra to four decimals. This is called a **numerical gradient check**

, and in session 14 it is how you will prove your hand-derived backpropagation is correct before you trust it. Remember that it exists.

WHY A NETWORK NEEDS IT

Your 2–3–1 network from Tuesday is a chain: the input produces hidden values, the hidden values produce an output, the output produces a loss. To know how much a weight in `W1` affects the loss, you must multiply the rates along that chain. That is what backpropagation is — the chain rule, applied layer by layer, backwards. You now have the tool. Session 14 does the work.

Two things I have not told you the truth about yet



Real surfaces are not bowls

x^2 has one minimum and you always find it. A real loss surface has many valleys, and gradient descent finds whichever one it happens to roll into. In practice this matters far less than it sounds — but it is why two identical training runs can end at different answers, and why session 16 asks you to fix a random seed.



Real problems have many dimensions

You descended one number. A network has thousands, and the slope becomes a *gradient* — one slope per parameter, all computed at once. The update rule is unchanged; it just applies to every entry of `W` simultaneously. NumPy does that in one line, which is the reason we spent two sessions on arrays.

LAB 6 · 16 MINUTES IN CLASS

Descend, and plot the path

7

Lab 6 is a separate handout — `CSC4315_Lab06_Descend_And_Plot.html`. You turn the five steps you just did on paper into a function, draw the walk down the curve, and then find the cliff edge of the learning

rate for yourself rather than taking my word for where it is.



Same rules, and one addition

Finish in the room, hand up after three minutes. The addition:

keep your `numgrad` function

. You will paste it into session 14 to check work you have not done yet, and it is the only piece of code from unit 1 that comes with you.

Before Friday — and a note on where you are

Unit 1 ends here. In six sessions you have gone from "what is a Python list" to a working optimisation algorithm. You can compute a layer, batch it, and walk downhill on a function. Everything needed to train something real is in your hands except one question, which is Friday's: **downhill on what?**

1. **Finish Lab 6**, all six parts.
2. **Be able to state, without notes:** the update rule, what η does, what happens when it is too large, and what your first action is when you see `nan`.
3. **Keep `numgrad`**. Session 14 needs it.



Session 10 — Friday 09 October, 11:30

Unit 2 begins. *The learning problem, and a line learned by descent.* Features, labels, training and test sets, and loss — the function we are actually going to descend. We fit a straight line to six points exactly, by solving it, so that you have seen a fitted model before you meet an iterative one — and then we throw the exact solution away and reach the same line by descending to it, using precisely what you did by hand today. No libraries from here until session 17.